

Manual Técnico — Tabulador de Respostas Abertas

Documento gerado automaticamente a partir do código-fonte em `jumppi/tabulador` · 24/09/2026

1. Visão geral e arquitetura

O Tabulador é uma aplicação Flask (Python) + JavaScript puro, sem build, que transforma respostas abertas de pesquisas de mercado em categorias fechadas prontas para análise (planilha final, codebook e cruzamentos). O princípio de design é **a IA propõe, a pessoa decide**: nenhum resultado sai sem que o pesquisador aprove explicitamente cada etapa (categorias e depois a classificação).

O programa é multi-projeto: um único código em `jumppi/tabulador/` atende vários clientes (hoje `sesi`, com planilha "plana" em `SESI_cat/`; antes também havia `assertiva`, formato SurveyMonkey, removido do app no commit de 24/09). Cada módulo do pipeline opera sempre sobre o "projeto ativo", que é trocado dinamicamente por `projetos.ativar()`.

Fluxo de dados, do bruto ao resultado:

```
planilha-fonte (.xlsx)
  | load.py (lê, limpa PII, tipa colunas, aplica filtros de base, deriva banners)
  v
output/base/{base.json, base.xlsx, variables.json}
  |
  | para cada pergunta aberta marcada "codificar":
  v
codeframe.py (preparar -> induzir com IA -> aprovar categorias)
  |
  v
coding.py      (codificar amostra/restantes com IA -> revisar/confirmar -> aprovar -> aplicar_na_base)
  |
  +--> supervisao.py / supervisor.py (segundo codificador de IA, aceite automático)
  |
  v
output/base/base.json (agora com <QID>_COD1/_COD2 e nomes)
  |
  +-> exportar.py      -> planilha final no formato da fonte (Q*_CAT / Q*_CAT2)
  +-> codebook.py      -> codebook.xlsx / .html
  +-> crosstabs.py     -> cruzamentos.xlsx (banners)
  +-> report.py        -> relatório HTML de conferência por pergunta + índice
```

Tudo isso é orquestrado por `app.py` (API Flask) e consumido por `static/app.js` (SPA sem framework). A camada de IA (`llm.py`, `llm_cli.py`, `simulador.py`) é abstraída atrás de uma única função (`llm.chamar_json`), de modo que o restante do pipeline nunca sabe se está falando com a OpenAI, Gemini, Groq, um programa de CLI instalado no computador ou o modo de teste simulado.

2. Modelo de projeto

`projetos.py` mantém o registro de projetos em `projetos.json`:

```
{"ativo": "sesi", "projetos": {"sesi": "../SESI_cat"}}
```

Cada projeto é uma pasta com `projeto.py` (módulo Python executado dinamicamente via `importlib`) ou `projeto.json` (declarativo, gerado pelo assistente "Novo projeto" e carregado por `novo_projeto/projeto_json.py`, que o converte num objeto com a mesma interface de um `projeto.py`). `projetos.ativar(slug)`:

1. carrega o módulo do projeto (com cache em `_modulos`);
2. aponta `config.PROJETO`, `config.PROJETO_PASTA`, `config.FONTE_XLSX`, `config.CONTEXTO_PROJETO` etc. para os valores do projeto;
3. resolve a pasta de saída (`OUTPUT_DIR`), desviando para `<saída>_teste` se o modo teste estiver ligado (copiando a base já lida na primeira vez, para não precisar reler a planilha);
4. chama `variables.ativar(mod)`, que faz `variables.py` (o "mapa canônico") passar a repassar `PERGUNTAS`, `FILTROS`, `DERIVADAS`, `BANNERS_PADRAO`, `PII_MATCH`, `BACKCODING`, `PLANO`, `SAIDA_PLANILHA` etc. do projeto ativo via `__getattr__` dinâmico.

Campos principais de um projeto (formato "plano", o único hoje em uso):

Campo	Papel
NOME / CLIENTE	identificação exibida na interface e nos relatórios
CONTEXTU_PROJETO	texto livre que entra em todo prompt da IA (quem respondeu, o que é a pesquisa)
FORMATO	"plano" (1 cabeçalho, 1 coluna por pergunta) ou "surveymonkey" (cabeçalho duplo, legado)
FONTE_XLSX	planilha-fonte (somente leitura)
PLANO	{aba, aba_codebook, id} — aba de dados, aba de enunciados, coluna de identificador
PERGUNTAS	lista de specs: id, tipo (meta/unica/aberta/numerica/multipla/grade_numerica), rotulo, coluna, base, niveis, codificar, instrucoes_frame, nps
FILTROS / FILTROS_DESCRICAO	funções df -> Series[bool] que definem a "base" de cada pergunta (quem foi elegível a respondê-la)
DERIVADAS / BANNERS_PADRAO	variáveis calculadas (faixas de NPS, mapas de valores, back-coding) usadas como colunas de corte nos cruzamentos
PII_MATCH	palavras-chave de nome de coluna que marcam dado pessoal (nunca entra na base)
BACKCODING	perguntas com frame FIXO (ver seção 12)
SAIDA_PLANILHA	{arquivo, aba, id, colunas} — como escrever a planilha final no formato do cliente
PERFIL	(opcional) mapeamento para a janela de "perfil de quem citou" (NPS, satisfação, retenção, grupo, atributos)
CONFERENCIA	(só em projeto.json) registro de quem confirmou o pedido do cliente — exigido pelo validador

No formato `projeto.json`, `FILTROS` é declarativo (regras {"pergunta", "entre"|"em"} ou {"todos": true}) e `novo_projeto/projeto_json.py` as compila em funções reais na carga.

3. Pipeline de dados (load.py)

`load.executar()` decide entre dois caminhos conforme `V.FORMATO`:

- `executar_plano()` (SESI e todo projeto novo): lê uma única aba com `pandas.read_excel`,

pega só as colunas declaradas em `PERGUNTAS` (colunas fora disso, se tiverem dado e não forem PII, geram aviso — evita perda silenciosa), tipa cada coluna (`_to_num` para numéricas, `limpar_texto` para texto), monta o DataFrame e valida que `respondent_id` não se repete.

- `carregar_aba()` / `executar()` (SurveyMonkey, legado): lê cabeçalho duplo (linha de pergunta + linha de opção), com `mapear_colunas()` casando cada coluna com uma `spec` de PERGUNTAS por normalização de texto (`v.norm`), tratando PII, campos "Outro. Qual?", itens de múltipla escolha (viram 0/1) e grades numéricas. Duas abas (autopreenchido/telefone) são unidas e deduplicadas por `respondent_id`.

Depois da leitura: `aplicar_bases()` transforma em NaN tudo que está fora do filtro de base da pergunta (para qualquer tabulação ingênua já usar o denominador certo); `derivar_banners()` calcula as variáveis derivadas (faixas, mapas ou back-coding); `salvar_base()` grava `output/base/{base.json, base.xlsx, variables.json}` — `variables.json` é o "registro de variáveis", com rótulo, tipo, base e origem de cada coluna, usado pelo codebook e pelos cruzamentos.

4. Codeframe — indução de categorias (`codeframe.py`)

Ciclo de vida de uma pergunta aberta, por `qid`:

1. `preparar(qid)`: extrai da base as respostas dentro do filtro de base, agrupa por texto idêntico (`chave_texto`, normalizado), detecta NS/NR por regex conservadora (`_NSNR_RE`), e monta `respostas.json` com uma entrada por resposta única: `{rid, chave, texto, n, respondent_ids, nsnr}`. O `rid` é estável: ao reler a base, a mesma resposta (mesma chave) mantém o `rid` de antes — isso é o que permite recarregar a planilha sem perder classificação (ver `_conciliar`, seção 11).
2. `induzir(qid)`: monta um prompt (`_prompt_inducao`) com o contexto do projeto, o enunciado, as instruções da pergunta e até `TABULADOR_MAX_INDUCAO` respostas (acima disso, usa todas as repetidas + amostra aleatória fixa das demais, semente 0 — determinístico). Pede à IA, via `llm.chamar_json` com `SCHEMA_FRAME`, uma lista de 6–15 categorias mutuamente exclusivas, cada uma com nome curto, definição e exemplos literais. O resultado vira `frame.json` com status "`rascunho`", mais as categorias fixas `Outros` (97) e `NS/NR` (98) (`FIXAS`).
3. **Edição humana** (antes ou depois de aprovar): `renomear`, `fundir`, `adicionar`, `remover`, e `refinar(qid, feedback)` — um segundo prompt (`SCHEMA_REFINO`) em que a pessoa escreve em português ("junte X e Y", "crie categoria para Z") e a IA devolve o frame revisado preservando `codigo_anterior` das categorias que continuam e `incorpora` para fusões; o código local então remapeia códigos removidos/fundidos em `codificacao.json` via `_remapear_codificacao` (o que estava numa categoria fundida passa a apontar para o destino; o que sobrou sem destino vai para Outros). Há `desfazer_refino` (volta para `frame_anterior.json`).
4. `aprovar(qid)`: trava o frame (`status = "aprovado"`). Importante: aprovar as categorias é uma etapa única, **antes da primeira classificação**; depois disso, editar o frame não trava mais a classificação (`_status_apos_edicao`) — passa a valer nas próximas chamadas à IA sem exigir nova aprovação formal.

5. Classificação (`coding.py`)

`codificar(qid, ...)`: para cada resposta única pendente, monta lotes de `config.LLM_LOTE` (padrão 30) respostas e dispara `config.paralelo()` chamadas simultâneas via `ThreadPoolExecutor` (`_chamar_lote`), cada uma pedindo à IA (`SCHEMA_CODIFICACAO`) a categoria primária (obrigatória), secundária (opcional), confiança (0–1) e justificativa, usando *somente* os códigos do frame aprovado. Respostas com comentário do pesquisador recebem uma instrução obrigatória extra no prompt. Parâmetros:

- `limite`: classifica só uma **amostra** (metade das mais frequentes + sorteio fixo do resto — `_amostra`) para validar antes de rodar tudo;
- `somente_faltantes`: mantém tudo já classificado e manda à IA só o que falta;
- `preservar_humano`: nunca reenvia o que já foi confirmado/corrigido por uma pessoa

(`_protegido` — origem `"humano"` ou `validado=True`).

Se um lote falha por limite/cota do provedor (`llm.e_limite_ou_cota`), os lotes ainda não iniciados são **cancelados** (não insiste) e ficam pendentes para a próxima tentativa, em vez de virarem erro.

Cada item de `codificacao.json` guarda: `primaria`, `secundaria`, `confianca`, `justificativa`, `origem` (`llm | llm+comentario | humano | regra | erro`), e opcionalmente `validado` / `validado_por` (`"humano"` ou `"auto"`), `comentario`, `corrigido_de` (quando uma correção humana muda o que a IA tinha posto — entra na taxa de acerto), `auditoria` e `ia_original` (ver seção 7).

Outras operações centrais:

- `atualizar_item`: correção manual de uma resposta (marca `origem="humano"`, `confianca=1.0`).
- `validar(qid, rids, valor)`: confirma/desconfirma em lote.
- `resumo_revisao(qid)`: calcula a **taxa de acerto da IA** = confirmadas ÷ (confirmadas +

corrigidas depois da IA), excluindo confirmações automáticas do supervisor (que não contam como "a pessoa validou").

- `recodificar`: reenvia só rids escolhidos (comentados, ou tudo que ainda não foi revisado/protegido — usado pelo botão "Reclassificar tudo").

- `exportar_revisao` / `importar_revisao`: ida e volta via Excel (`revisao.xlsx`), para quem prefere revisar fora da interface.

- `aprovar(qid)`: exige que não falem respostas por classificar; trava `status="aprovado"`.
- `aplicar_na_base()`: para cada pergunta com codificação aprovada, grava `<QID>_COD1/_COD2` (código) e `_NOME` (rótulo) na base, e recalcula os banners de back-coding.

- `recarregar_base()`: relê a planilha-fonte preservando rids estáveis, reabre para revisão perguntas aprovadas que ganharam respostas novas, e registra a versão em `base/versoes.json`.

6. Camada de IA

`llm.py`

Função central `chamar_json(system, user, schema, nome)`: registra cada chamada (sucesso ou erro) em `output/llm_log/<timestamp>_<nome>.json` (prompt, resposta, tokens, tempo) — é a fonte de dados da aba Consumo e das estimativas de tempo (`progresso.tempo_medio`). Compatível com qualquer provedor estilo OpenAI (só muda `OPENAI_BASE_URL`): tenta `json_schema` estrito, cai para `json_object`, e por fim para texto livre com extração de JSON (`extraair_json`, inclusive dentro de blocos ``json``). Roteia automaticamente para `simulador.py` (modo teste) ou `llm_cli.py` (provedores `cli_*`) antes de tentar a API HTTP. Tem um gerenciador de contexto `usando(provedor, modelo)` que troca temporariamente o provedor (usado pela auditoria, que pode rodar num modelo diferente do principal, sem alterar o `.env`). `mensagem_amigavel()` e `diagnosticar()` traduzem erros comuns (sem crédito, chave inválida, limite de taxa, modelo inexistente, timeout) para português, usados no botão "Testar conexão" e nas mensagens de erro da interface.

`llm_cli.py`

Permite usar um programa de IA já instalado e logado no computador (Claude Code, Codex, Gemini CLI, Antigravity) sem chave de API: abre o programa em modo não interativo, sem ferramentas, numa pasta temporária vazia, e extrai o JSON da saída. Só Claude Code e Antigravity estão testados. Limitado a no máximo 2 chamadas simultâneas (`config.paralelo()`).

`simulador.py` (modo teste)

Responde no lugar da IA de forma **determinística** e sem rede: agrupa palavras frequentes das respostas em categorias (indução), interpreta "junte/mescle", "crie" e "remova" citando nomes de categoria (refino), e classifica por sobreposição de palavras-radical entre a resposta e nome/definição/exemplos da categoria (com Outros quando não há nenhuma em comum). Usado para treinar e demonstrar o app sem gastar cota; grava em `<saída>_teste`.

`usage.py` (aba Consumo)

Consolida todos os arquivos de `output/llm_log/`, junta tokens/tempo por chamada, estima custo em US\$ (tabela `PRECOS`, US\$/1M tokens) e converte para R\$ (`TABULADOR_USD_BRL`). Agrupa por modelo, por pergunta/operação, por dia e por hora, e expõe o limite diário aproximado dos planos gratuitos (`LIMITES_DIARIOS`, ex.: Gemini = 20/dia) comparado ao uso de hoje.

`progresso.py`

Estado global thread-safe de uma "operação longa" em curso (etapas com peso, feitos/total, velocidade, tempo restante, lotes, chamadas de IA ativas, últimos eventos). `app.py` abre uma operação com o gerenciador de contexto `progresso.operacao(titulo, etapas)`; os módulos do pipeline avisam progresso com `progresso.etapa()` / `progresso.atualizar()` / `progresso.evento()`. A interface faz polling em `/api/progresso`. `tempo_medio(prefixo)` estima a duração de uma chamada única (indução, refino) a partir do histórico do mesmo tipo no `llm_log`.

7. Supervisão / segundo codificador

`supervisao.py` implementa dois mecanismos, usados pela interface e por `supervisor.py` (CLI):

- `auditar(qid, provedor, modelo, escopo)`: pede a um SEGUNDO modelo (escolhido na hora, pode ser diferente do principal, via `llm.usando`) que confira cada resposta já classificada (`SCHEMA_AUDITORIA`). Grava em cada item `auditoria = {por, ok, em, [primaria, secundaria, nota]}`. Quando discorda, a interface mostra "auditor sugere: X" (botão/tecla **S** para aceitar).
- `auto_aceitar(qid, limiar, exigir_auditoria)`: confirma sozinho (`validado_por="auto"`) o que tem confiança $\geq$ limiar (padrão 0,85) e a auditoria concordando (se exigida), nunca mexendo no que uma pessoa já conferiu. `desfazer_auto` reverte. Confirmações automáticas **não** entram na taxa de acerto da IA (só conferências humanas contam) — ver `resumo_revisao`.
- `conferencia()`: separa contagens de quem conferiu o quê (pessoas, automático, sugestões do auditor adotadas) — usado na aba Resultados e Consumo.

`supervisor.py` é a mesma lógica pela linha de comando, pensada para "matar o máximo com alta confiança e deixar o resto para o pesquisador": `status`, `rodar` (classifica com retentativas), `reclassificar` (por categoria/confiança), `auditar/aplicar-auditoria` (fluxo em lote via TSV/JSON, útil quando o auditor é um humano ou uma IA externa ao app), `adotar-sugestoes` (troca a classificação pela sugestão do auditor, mantendo `ia_original` e ficando pendente para confirmação humana), `auto-aceitar`, `desfazer-auto`, `fila` (lista as pendentes).

`perfil.py` monta a "janela de revisão por categoria": para cada categoria, compara quem a citou com a base inteira da pergunta em indicadores configuráveis (`PERFIL` no projeto) — NPS (média/promotores/detratores), satisfação em escala, risco de retenção, distribuição por grupo (ex.: escola) e atributos onde quem citou avalia pior que a média. Serve para o pesquisador decidir rapidamente "essa categoria faz sentido e bate com o que já sei sobre esses respondentes?" e tem "Confirmar N pendentes" em bloco (nunca inclui itens em que o auditor discordou).

`resumo_geral.py` gera o panorama do projeto inteiro (aba Resultados): junta os temas mais citados e trechos de exemplo de todas as perguntas **aprovas**, e mantém em cache uma narrativa de um parágrafo escrita pela

IA (só recalculada quando pedido, para não gastar chamadas à toa).

8. Saídas

- `exportar.py`: só roda se o projeto define `SAIDA_PLANILHA`. Copia a aba da planilha-fonte

(nunca a altera) e escreve, por respondente, o nome da categoria principal na coluna declarada (`Q*_CAT`) e a secundária numa coluna nova (`Q*_CAT2`); acrescenta uma aba `Categorias` com o frame e a situação de cada pergunta. Só entram perguntas com classificação **aprovada**.

- `codebook.py`: junta o registro de variáveis (`variables.json`) com a base processada e gera

quatro abas: `Perguntas` (uma linha por variável, com base e n), `Valores` (distribuição de cada categoria/item/faixa numérica), `Derivadas` (regra de cada banner) e `Code frames` (categorias de cada aberta codificada). Também gera uma versão HTML de leitura rápida.

- `crosstabs.py`: para cada bloco tabulável (fechada, múltipla, numérica/NPS, grade, aberta

codificada, derivada), monta uma tabela de % vertical por banner (colunas = níveis dos banners escolhidos, padrão `BANNERS_PADRAO`), com base de coluna, alerta de base pequena (`MIN_BASE=10`), cálculo automático de NPS quando marcado, e uma aba "Geral" que junta tudo numa única tabela longa (mais uma aba `dados_longos` para reuso em outra ferramenta).

- `report.py`: relatório HTML autocontido por pergunta (sem dependências externas): frame com

contagens, gráfico de menções por categoria, histograma de confiança, tabela de alertas (confiança baixa, Outros, erro de LLM) e a tabela completa filtrável/ordenável em JS puro. Gera também `perguntas/index.html`, o índice de todas as perguntas com status de frame/codificação.

9. Nuvem (`nuvem.py`, Turso/libSQL)

Prova de conceito de armazenamento compartilhado: os mesmos arquivos que `codeframe._ler/_gravar` gravariam em disco (respostas, frame, codificação, instruções) viram uma linha na tabela `blobs` (chave `projeto+qid+arquivo`) de um banco Turso (libSQL) remoto. `nuvem.ativa()` liga sozinho quando `TABULADOR_TURSO_URL` está definido (tipicamente herdado do `.env` compartilhado da equipe no SharePoint) — testes automatizados, validação de projeto novo e modo teste **nunca** tocam o banco real (só um banco local `file:...`, se configurado). Todo `codeframe._ler/_gravar/_existe/_remove` verifica `nuvem.ativa()` primeiro e delega para `nuvem.ler/gravar/existe/remove` em vez do sistema de arquivos — o resto do pipeline não sabe a diferença. Limitação assumida: **uma pessoa por pergunta por vez** (a última gravação vence, sem merge). CLI própria (`python nuvem.py -p sesi status|backup|subir`) para diagnosticar, fazer backup e subir uma pasta `output/perguntas` que existia só em disco.

10. App web (`app.py` + `static/app.js`)

`app.py` é uma API Flask fina: quase toda a lógica de negócio já mora nos módulos acima; as rotas só validam entrada, seguram um lock global (`_lock`, uma escrita por vez, porque o estado é arquivo/JSON) e devolvem `_payload_pergunta(qid)` — um payload único (frame + codificação + tabela + resumo de revisão + conferência) reaproveitado por quase todo endpoint de pergunta, para o front-end sempre re-renderizar a partir de uma fonte só. Endpoints por área:

Área	Rotas
Painel/status	GET /api/status, POST /api/projeto, GET /api/usage, GET /api/progresso
Configurar IA	POST /api/config, POST /api/config/testar, POST /api/modo-teste[/limpar]

Área	Rotas
Pergunta — frame	GET /api/pergunta/<qid>, POST .../preparar, .../instrucoes, .../frame/induzir, .../frame/categoria, DELETE .../frame/categoria/<codigo>, .../frame/fundir, .../frame/aprovar, .../frame/refinar, .../frame/desfazer
Pergunta — codificação	POST .../codificar, .../validar, .../item/<rid>, .../recodificar, .../aprovar, .../reabrir
Supervisão	GET .../perfil, POST .../auto-aceitar, .../desfazer-auto, .../auditar
Resultados	GET /api/resultados, POST /api/gerar/<tipo>, /api/gerar-todos, GET /api/variaveis-cruzamento, /arquivo/<tipo>
Resumo geral	GET /api/resumo-geral, POST /api/resumo-geral/gerar
Base nova	POST /api/load (recarrega a planilha preservando trabalho)
Gerenciar projeto / zona de perigo	GET /api/perigo, POST /api/perigo/pergunta, /api/perigo/projeto
Novo projeto	POST /api/novo/iniciar, /api/novo/aba, /api/novo/validar, /api/novo/sugerir, /api/novo/criar
Infra	GET /, /relatorio/<qid>, /revisao/<qid>.xlsx, /api/versao, POST /api/abrir-pasta

A "zona de perigo" nunca apaga de verdade em modo arquivo local: move para `output/_lixeira/<data>_<o quê>/`; no modo nuvem, os JSON (frame/codificação) são removidos direto do banco (sem lixeira lá), mas `revisao.xlsx/relatorio.html` (sempre locais) continuam indo para a lixeira. Excluir um projeto nunca apaga a pasta do projeto nem a planilha-fonte.

`app.py` também resolve o problema de "clique duplo abre outra janela": calcula uma `VERSAO` (hash dos arquivos do programa) e, ao iniciar, verifica se já há um processo na porta 5000 — se for a mesma versão, só abre o navegador; se for antiga, mata o processo antigo (`_encerrar_antigo`, via `taskkill`, só se confirmar que é um `app.py` do Tabulador) e sobe a versão atual.

`static/app.js` (SPA de ~1.600 linhas, sem framework, roteada por `location.hash`): mantém um `state` global único; principais blocos (por comentário de seção no próprio arquivo): infra de chamada à API com indicador de progresso (`api()`, `busy()`, `renderProgresso`), painel geral, tela de pergunta (frame, ações de classificação, tabela de revisão com atalhos de teclado — setas, Enter confirma e sobe a próxima linha para o lugar da confirmada, `C` comenta, `1–9` troca categoria), janela de revisão por categoria (perfil de quem citou), auditoria por outra IA, barra de aprovação, aba Resultados (gráficos de categoria em CSS puro, sem biblioteca), aba Gerenciar projeto (zona de perigo com confirmação digitada), modal Configurar IA (troca de provedor, teste de conexão), aba Consumo (gráficos de uso/custo por período, com `localStorage` só para lembrar a preferência de agrupamento), modo teste, e o assistente completo de "Novo projeto" (edição da tabela de colunas do rascunho, sugestão da IA, validação, criação).

11. Base nova (recarregar planilha)

Ponto central: `coding.recarregar_base()` chama `load.executar()` de novo e, para cada pergunta já preparada, `codeframe.preparar()` — que reaproveita `rids` pela `chave_texto` (texto normalizado).

`_conciliar()` então: marca como removidas as respostas que sumiram (guardadas em `codificacao["removidos"]`, nada se perde de fato), casa pelo `rid` a classificação existente com a resposta

relida (levanta erro se a chave gravada não bate mais — proteção contra corrupção silenciosa), e reabre para revisão (`status` volta a "rascunho") qualquer pergunta **aprovada** que ganhou respostas sem classificação. Cada leitura fica registrada em `output/base/versoes.json` com o resumo de mudanças por pergunta.

12. Back-coding (`backcoding.py`)

Fluxo especial para fechar os campos "Outro. Qual?" de perguntas fechadas (ex.: Cargo, Segmento) num banner. `variables.BACKCODING` define um **frame fixo** (não gerado por IA) por banner; `codeframe.frame_fixo()` monta esse frame já **aprovado**; a codificação das respostas "Outro" segue o fluxo normal de `coding.codificar`. `backcoding.aplicar()` aprova e chama `aplicar_na_base()`, que usa os códigos resultantes para reconstruir o banner (`derivar_banners` com `backcodes`). CLI: `python run.py backcode [aplicar]; conferir()` mostra quantos respondentes ainda ficaram em "Outro" (deveria ser zero depois do processo).

13. Criação de projeto novo (`novo_projeto/`)

Processo desenhado para minimizar regra escrita à mão e travar em etapas verificáveis:

1. `perfil.py`: perfila a planilha-fonte com a MESMA limpeza do pipeline (`load.limpar_texto`, `load._to_num`), para o rascunho e o validador enxergarem exatamente o que o Tabulador vai ver — tipo de cada coluna, valores distintos (até 60), suspeita de dado pessoal por nome de coluna ou por conteúdo (regex de e-mail/CPF/IP/telefone), cor de fundo do cabeçalho (pista de quais colunas o cliente destacou como `_CAT` a categorizar) e enunciados de uma aba "Codebook", se existir.
2. `rascunho.py`: gera um `projeto.json` **sem IA**, por regras fixas e determinísticas: classifica cada coluna (`meta/unica/aberta/numerica/pii/saida/vazia`), copia as opções de fechadas literalmente, identifica NPS (inteiro 0–10), deduz o filtro de base de cada aberta comparando quem respondeu com as fechadas anteriores (`deduzir_filtro` — testa se um valor de uma fechada "explica" $\geq 90\%$ de quem respondeu a aberta, com folga de contaminação cruzada), e decide `codificar=True` pela cor da coluna `_CAT` correspondente. Tudo que não dá para inferir dos dados (contexto da pesquisa, confirmação do pedido) fica marcado com o sentinela "PREENCHER"; deduções que merecem checagem humana viram `_notas`.
3. `assistente.py`: funções que `app.py` expõe para a tela do assistente (`iniciar`, `sincronizar` as colunas de saída com as perguntas marcadas, `salvar`, `sugerir` — pede à IA configurada um rascunho de `CONTEXTO_PROJETO`, rótulos curtos e instruções por pergunta, sem inventar fatos —, e `criar`, que registra e já roda a primeira leitura da base).
4. `validar.py`: roda o **pipeline real** (`load.executar`) numa pasta/slug temporários e compara com a leitura direta da planilha para garantir conservação de dados — nenhum "PREENCHER" restante, colunas declaradas existem, id único/inteiro/sem vazio, valores fora das opções declaradas, resposta perdida por filtro ou conversão numérica, dado pessoal indevidamente declarado como pergunta, `DERIVADAS/BANNERS_PADRAO` consistentes, e a seção `CONFERENCIA` (quem confirmou o pedido, quando, e se a lista de abertas pedidas bate com as marcadas `codificar=True`). Erros bloqueiam o registro; avisos só pedem ciência.
5. `registrar.py`: só grava em `projetos.json` se `validar()` passar; nunca sobrescreve um projeto existente sem `forçar`.
6. `cli.py`: os mesmos passos por linha de comando (`python -m novo_projeto.cli perfil|rascunho|validar|registrar`), usados também pela *skill de IA* `.claude/skills/novo-projeto/SKILL.md` (*documentada em* `AGENTS.md` para Codex/Antigravity/Gemini).

14. Testes

tests/ (roda com `python -m unittest discover -s tests -v`; escreve em pastas temporárias via `TABULADOR_OUTPUT`, nunca chama a IA de verdade — `llm.set_cliente` injeta um fake):

- `test_sesi.py` — pipeline do projeto SESI ponta a ponta.
- `test_base_nova.py` — recarga de planilha preservando classificação (rids estáveis, conciliação).
- `test_modos_teste.py` — modo teste (simulador, isolamento de pasta `_teste`).
- `test_novo_projeto.py` — perfil/rascunho/validação/registro do assistente de projeto novo.
- `test_cli.py` — comandos de `run.py` / `novo_projeto/cli.py`.
- `test_ia_usage.py` — consolidação de consumo/custo (`usage.py`).
- `test_nuvem.py` — armazenamento em nuvem (Turso), isolado de qualquer banco real.
- `test_supervisao.py` (novo em 24/09) — auditoria por segunda IA e aceite automático por confiança.

15. Segurança e dados

- **Nunca versionados** (`.gitignore`): `.env` (chaves de API), `SESI_cat/data/` (planilha do cliente) e qualquer pasta `output*` (respostas, classificações, resultados).
- **PII**: colunas cujo nome bate com `PII_MATCH/PII_PALAVRAS`, ou cujo conteúdo tem cara de e-mail/CPF/IP/telefone, nunca entram na base, nos prompts nem em nenhuma saída — **exceto** a planilha final, que é uma cópia da própria planilha-fonte do cliente (já continha esses dados).
- **Chave de IA**: gravada só em `tabulador/.env` local; cada provedor guarda a própria chave (`TABULADOR_CHAVE_<PROVEDOR>`), nunca é devolvida pela API (`_ia()` só informa `tem_chave`).
- **Zona de perigo**: toda ação destrutiva pede confirmação digitada e vai para lixeira (local) ou é removida do banco (nuvem, sem lixeira lá — mitigado por `nuvem.py backup`).
- **Auditoria**: toda chamada à IA é logada (`output/llm_log/`) com prompt e resposta completos, para rastreabilidade; só perguntas **aprovadas** entram nos resultados finais.